University of BRISTOL
Centre for Academic Language and Development

Foundations of Computer Programming - LANG00052

Week 02/03

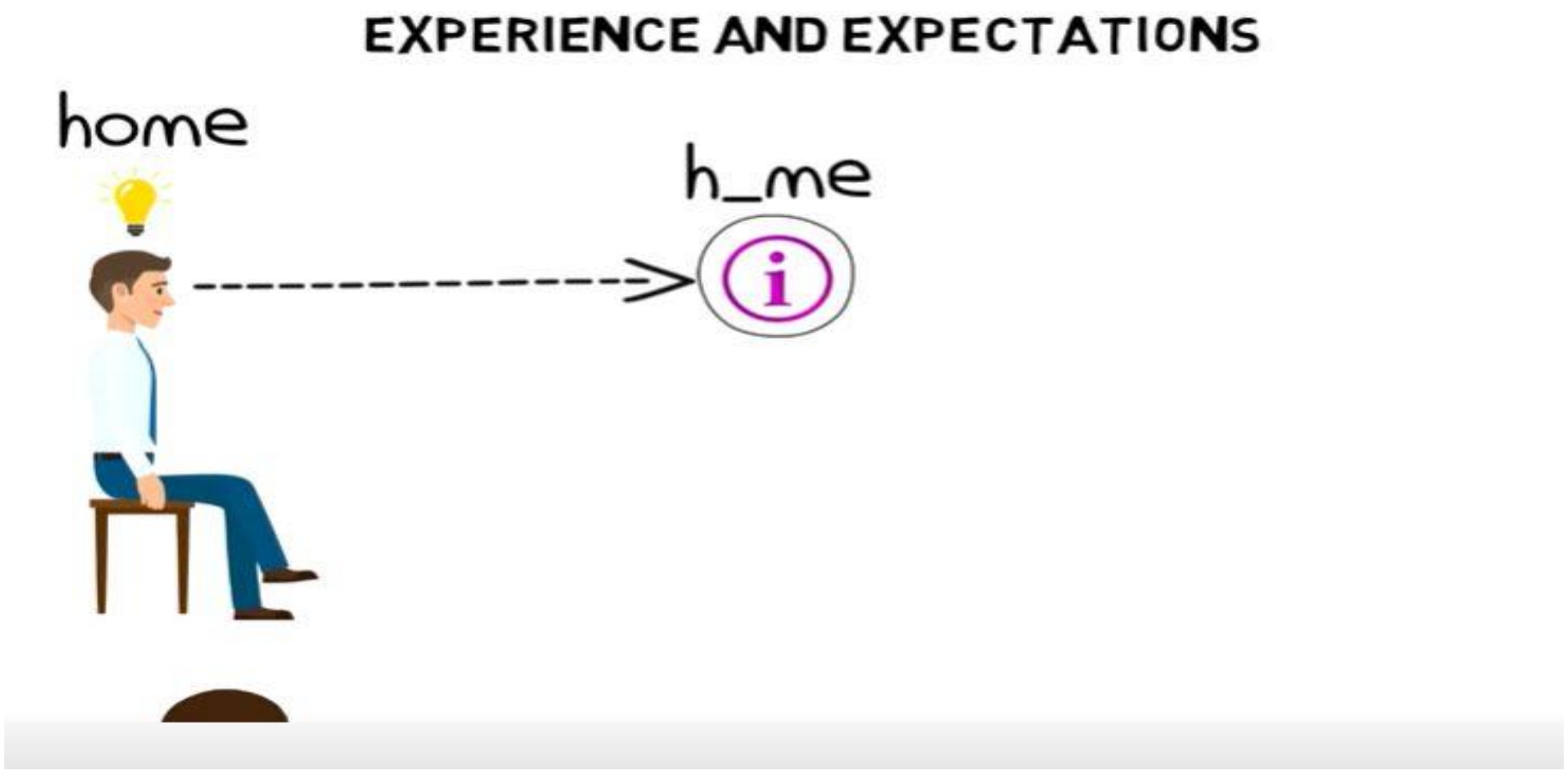
Problem solving

Syed Murad

Learning outcomes

- Understand the top-down and bottom-up approaches to problem-solving.
- Understand the advantages and disadvantages of top-down and bottom-up approaches.
- Recognise the computational thinking concepts and how to apply them.

Top-Down Approach



Top-Down Approach to software Development

In order to understand software systems, a developer needs to recognise what software is at both its largest and smallest levels, essentially acknowledging the different components that make up a system at different levels.

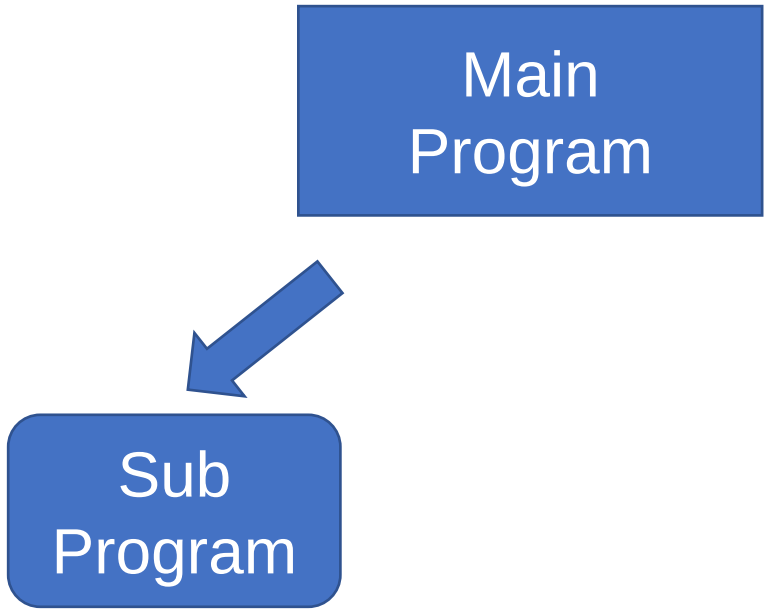
Top-Down Approach to software Development

- In this approach We focus on breaking up the big problems into smaller program.
- If the sub program is difficult, further we will break it into smaller program.
- Mainly this is used by structured programming languages like C, Fortan,COBOL etc.

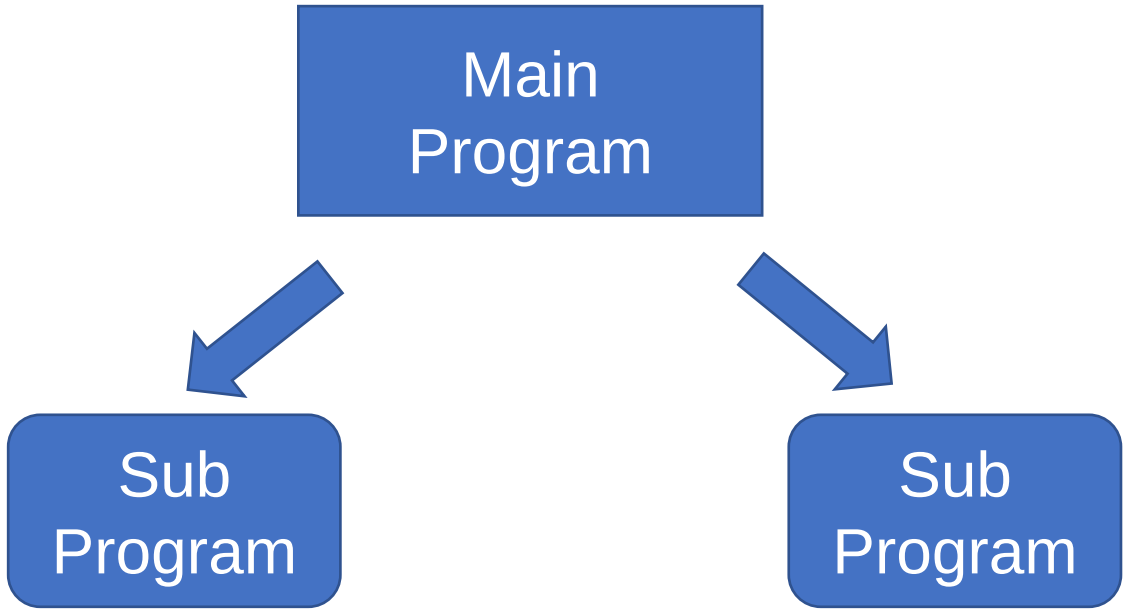
Top-Down Approach



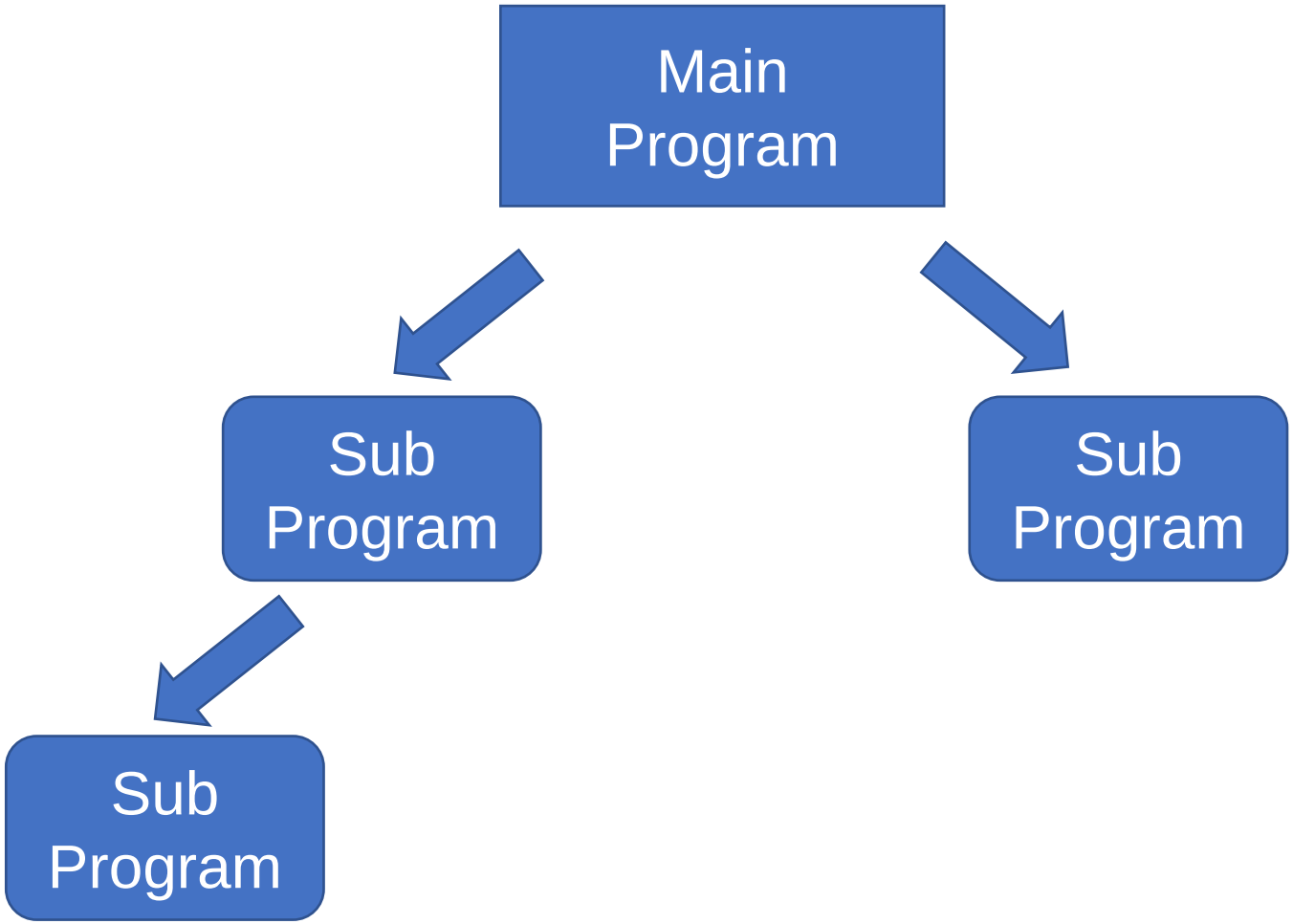
Top-Down Approach



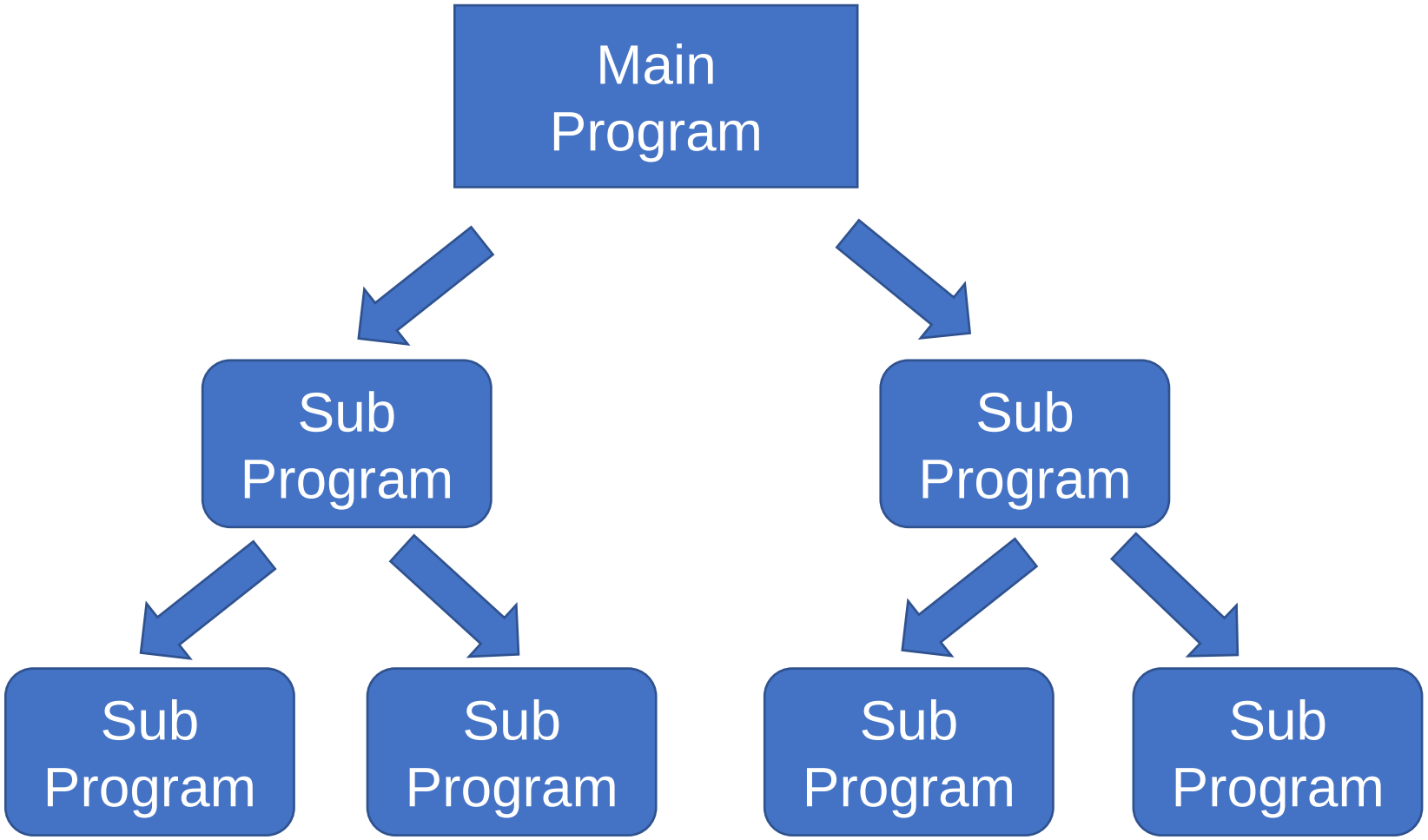
Top-Down Approach



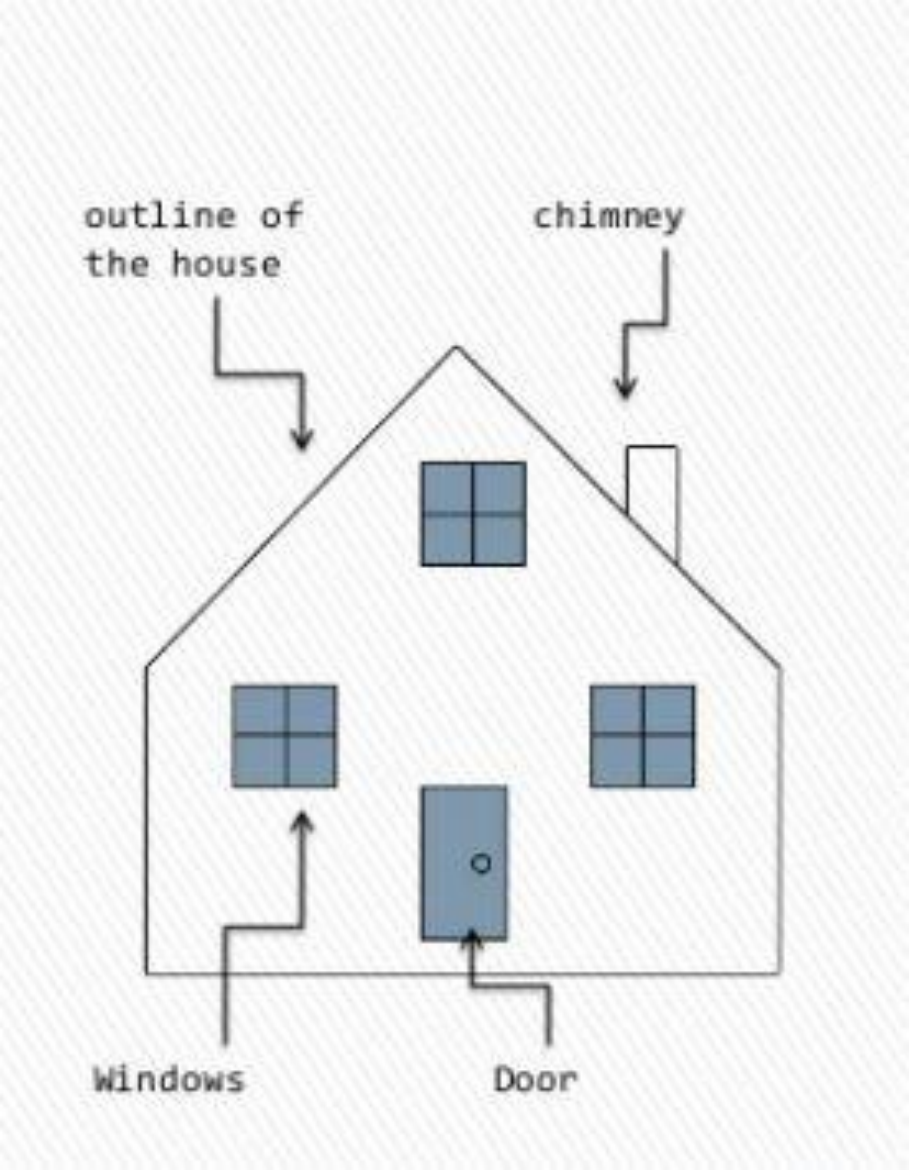
Top-Down Approach

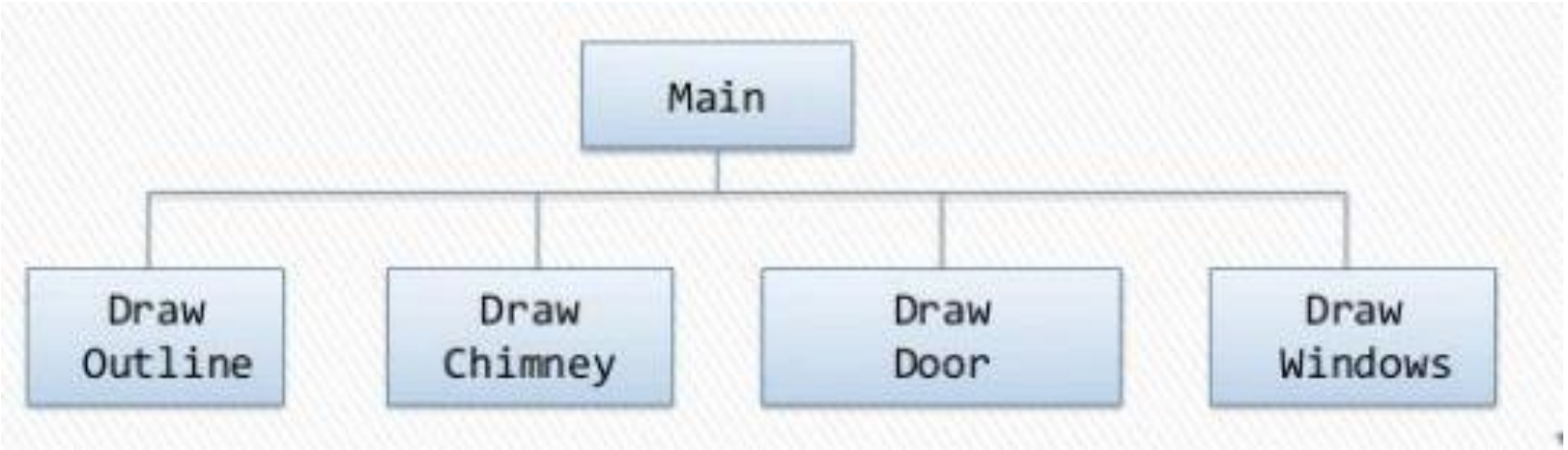


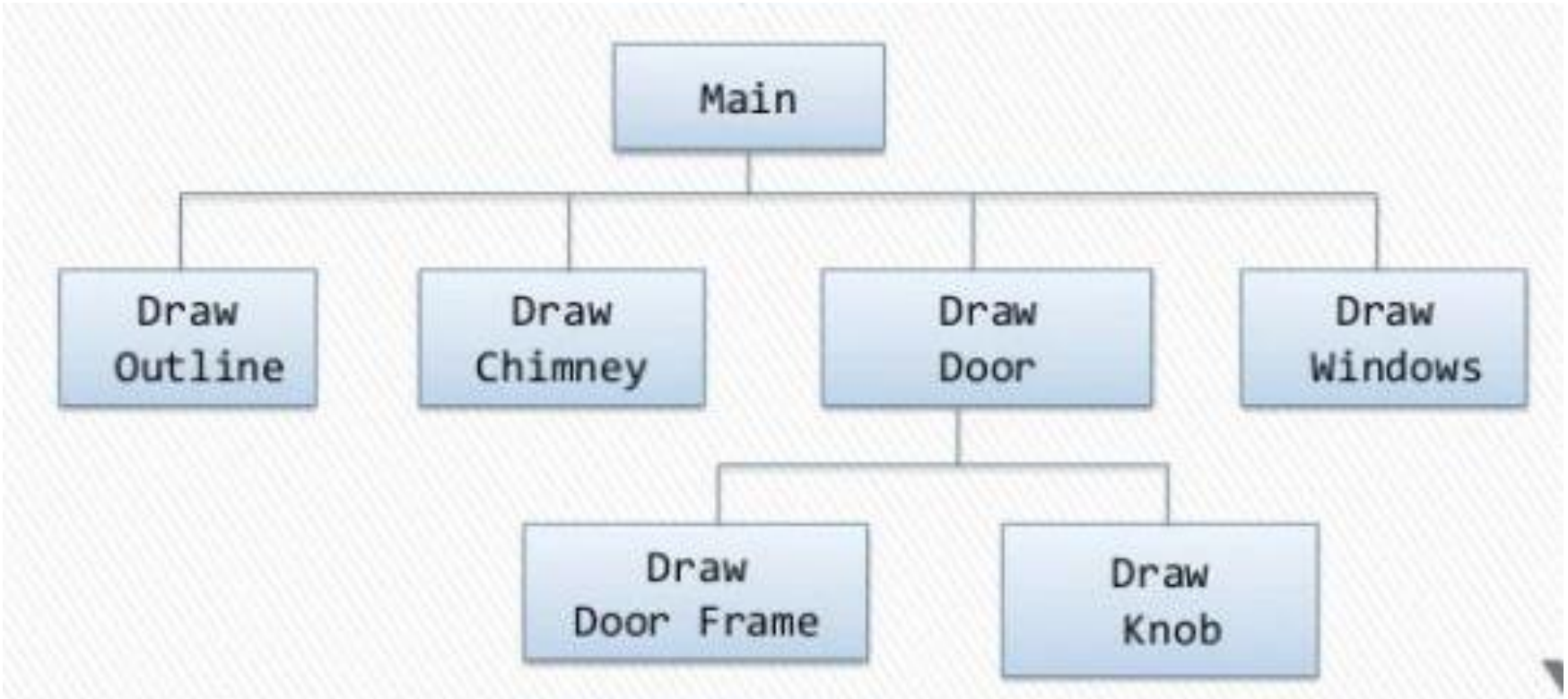
Top-Down Approach



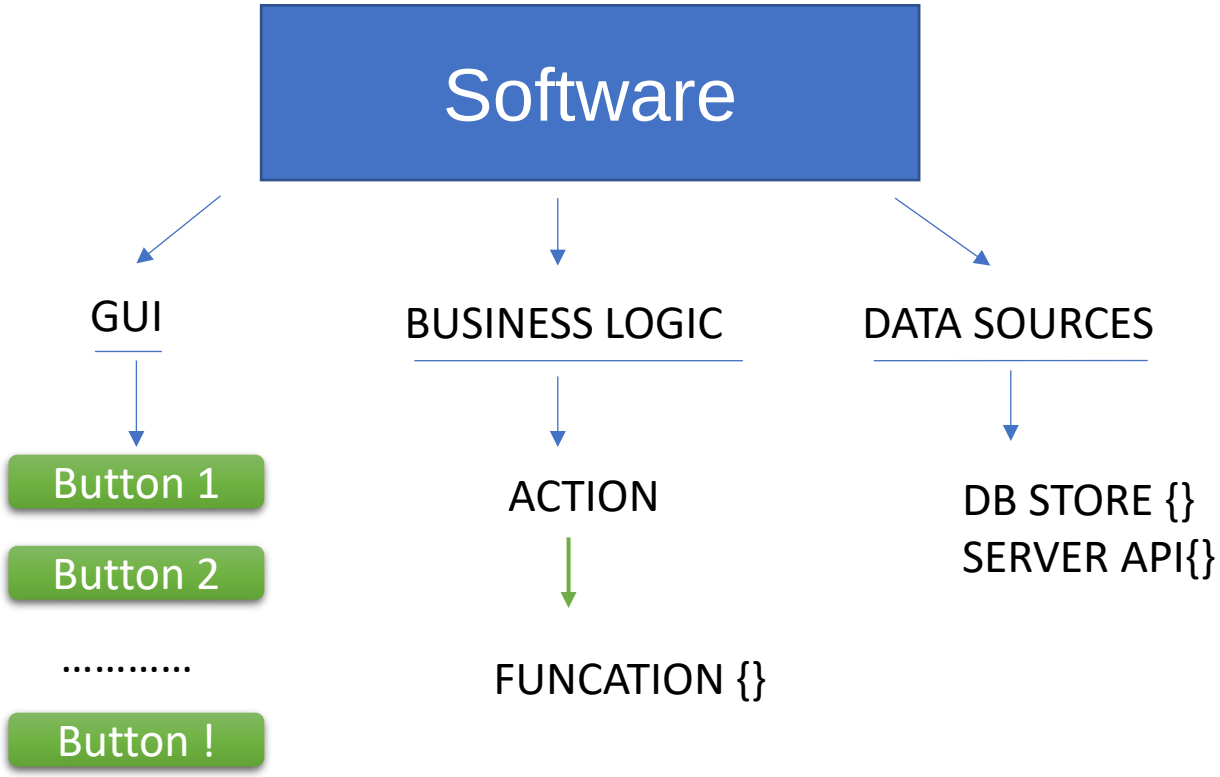
Write a program that draws this picture of a house.




Main
Draw Outline
Draw Chimney
Draw Door
Draw Windows


Main
Draw Outline
Draw Chimney
Draw Door
Draw Windows
Draw Door Frame
Draw Knob

Top-Down Design Approach



Advantages of top-down approach

- Provides a clear system overview.
- Simplifies problem-solving by breaking down the system.
- Improves organization and structure.
- Enhances communication and team collaboration.
- Facilitates better control and monitoring.
- Promotes reusability of components.
- Ensures effective resource allocation.
- Leads to easier maintenance and modification.

In a Nutshell

Top-down programming starts at the top and then works its way down.

So, you start at the most abstract level, by defining the problem, and from there add more details. It’s a bit like starting with an outline and then filling things in as you go until you have a story.


Bottom-Up Approach

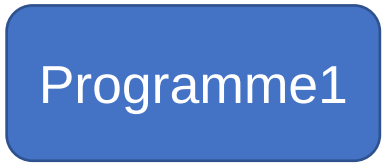
Bottom-Up Approach

- In bottom-up approach we will create small problems first, then we will solve the smaller problems.

- Then we will integrate all the small programs into a whole and complete the solution.

- Mainly used by objects-oriented programming languages like C++, Python, C# etc.

Bottom-Up Approach



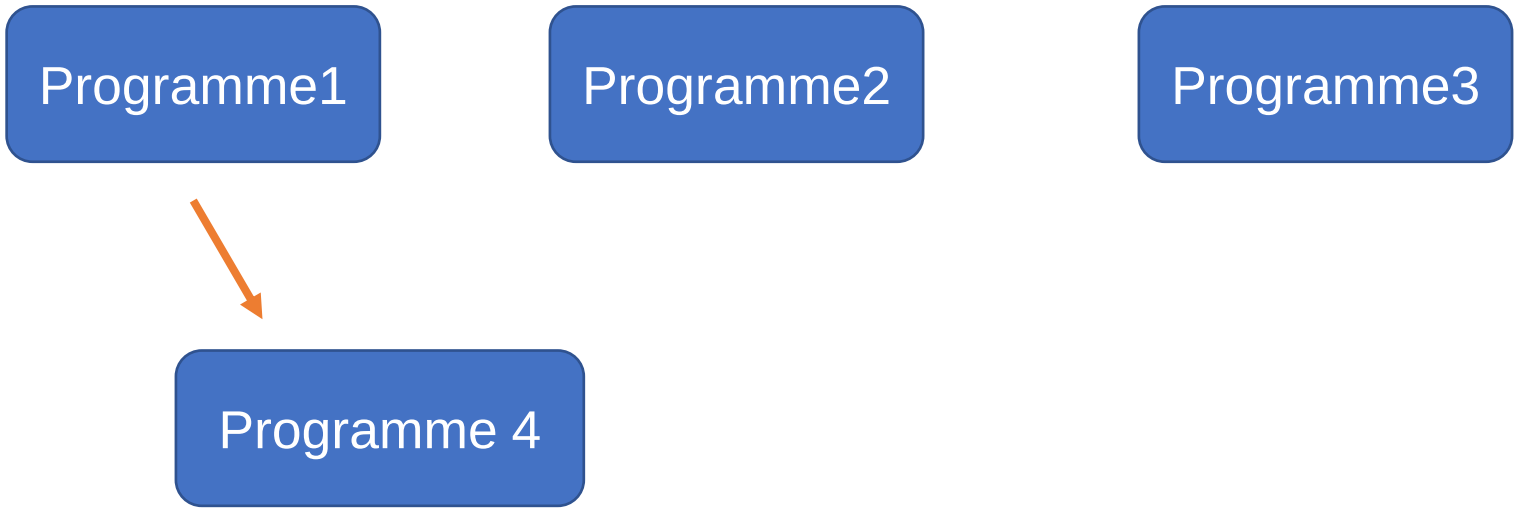
Bottom-Up Approach



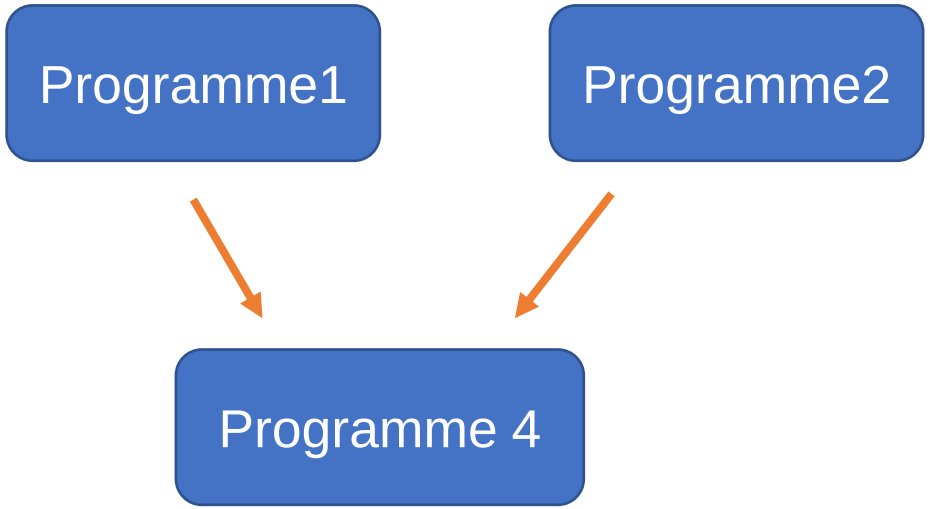
Bottom-Up Approach



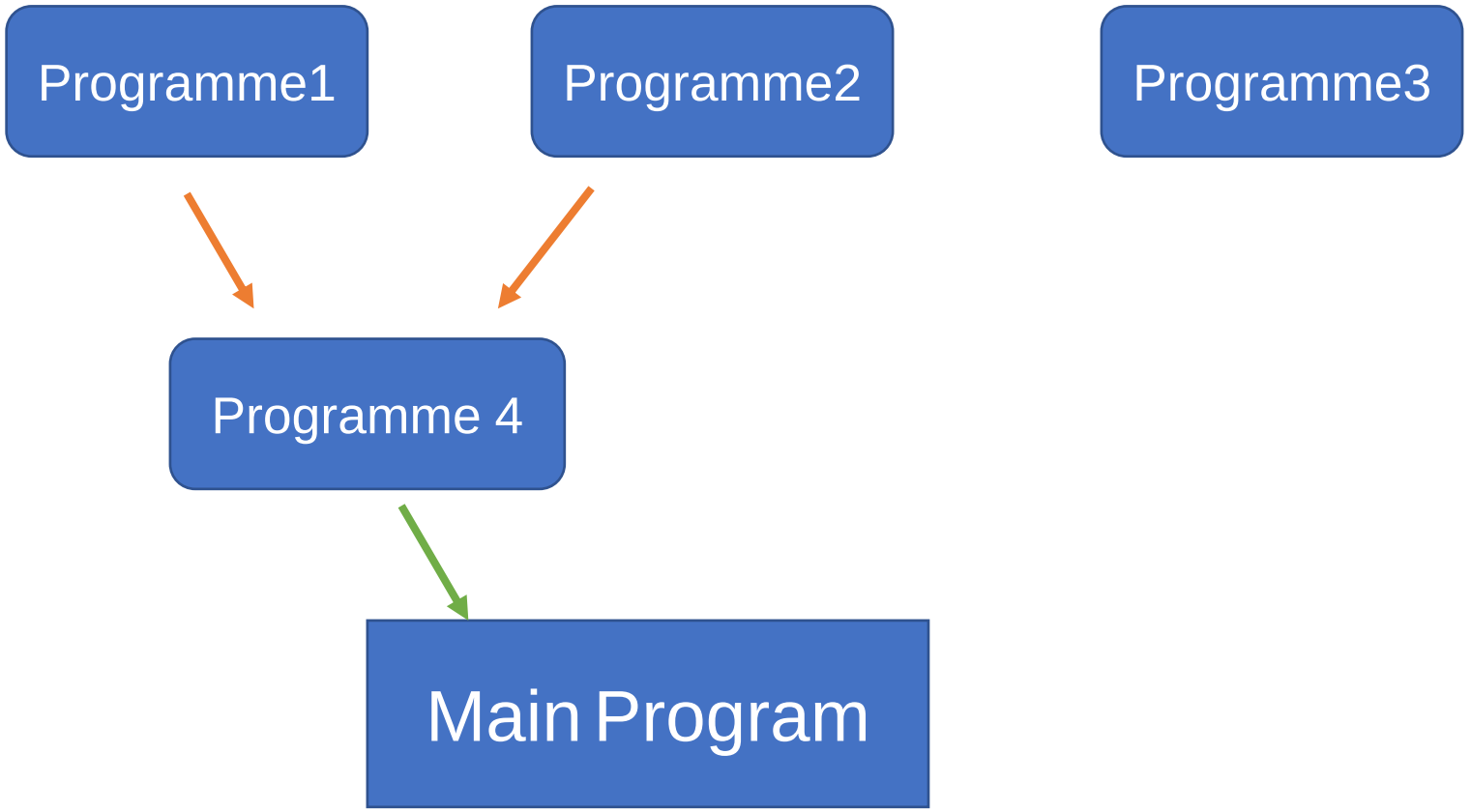
Bottom-Up Approach



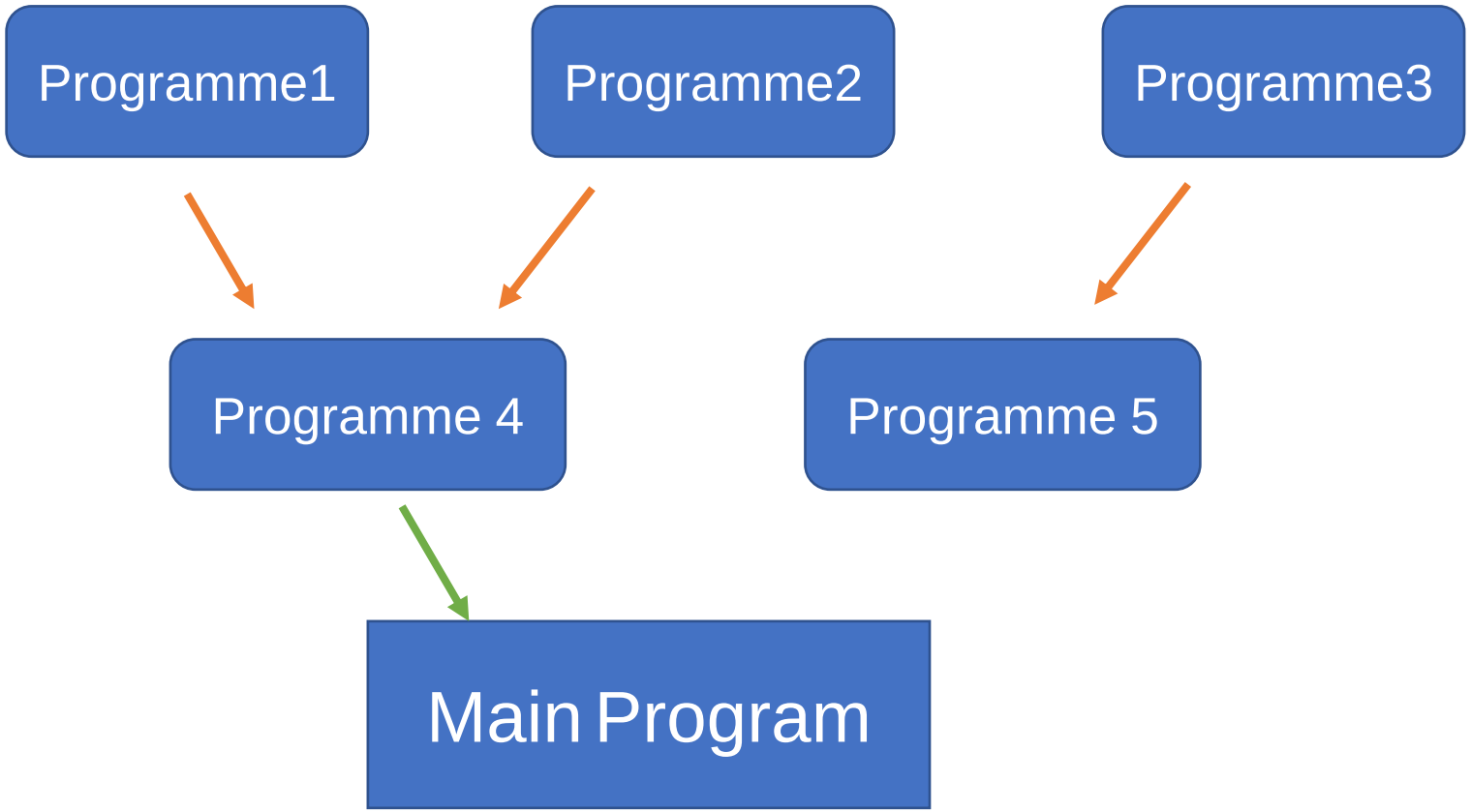
Bottom-Up Approach



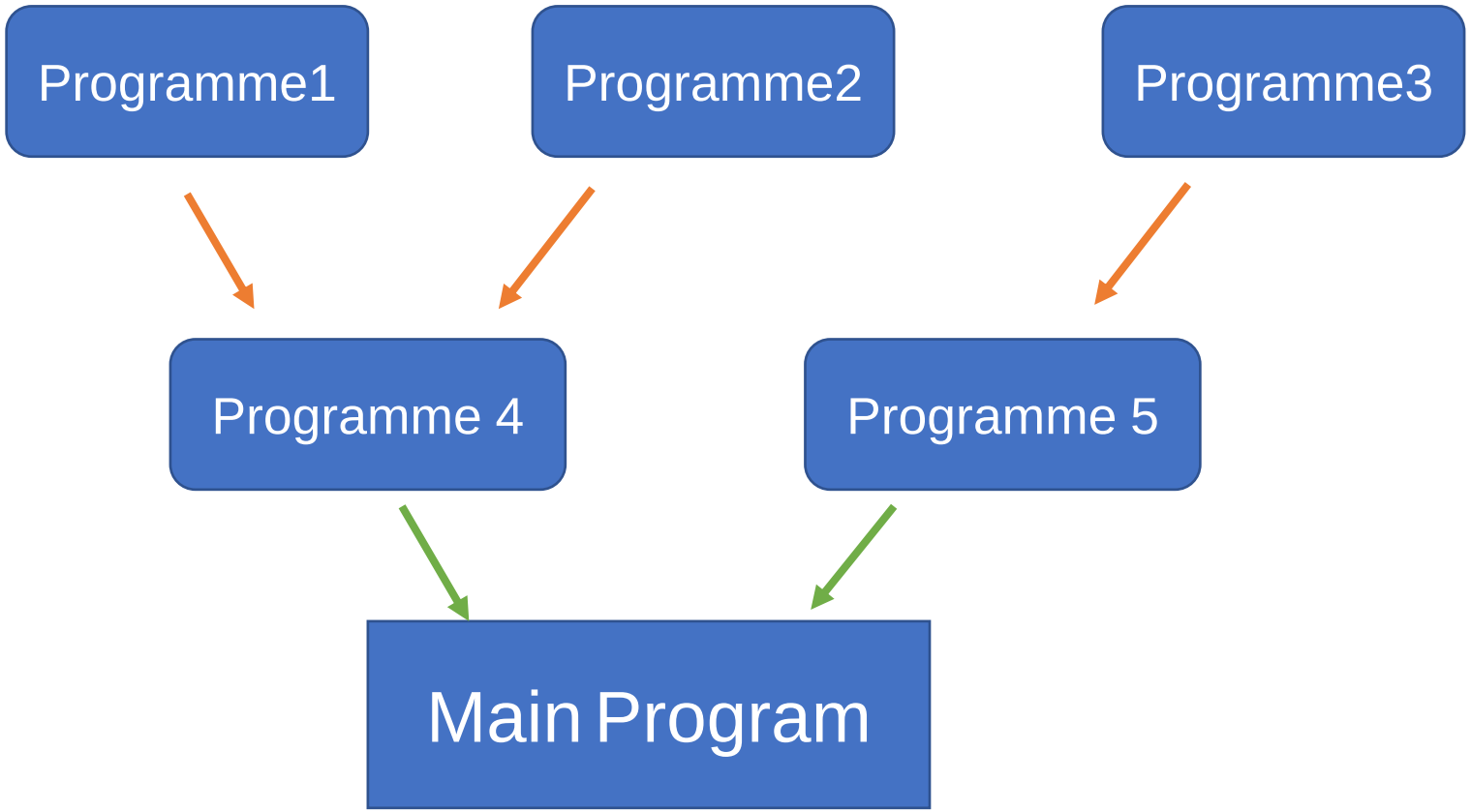
Bottom-Up Approach



Bottom-Up Approach



Bottom-Up Approach



Modularisation Approach

Modularisation simply means is breaking down a program into little packages or smaller packages.

We can call these packages, methods or functions or subroutines or procedures.

Different programming language has different terminology for that.

Modularisation Approach

An algorithm for prime number as an example:

```
PROGRAM CheckPrime:
    Read A;
    B  <- A  - 1;
    IsPrime <- TRUE;
    WHILE   (B    != 1)
      DO  IF  (A/B gives no remainder)
            THEN  IsPrime <- FALSE;
        ENDIF;
     B  <-  B  - 1;
    ENDWHILE;

IF (IsPrime  = FALSE)
        THEN Print “Not Prime”;
        ELSE  Print “Prime”;
   ENDIF;
END .
```

The first part checks if it’s prime

The second part prints out the result

Modularisation Approach

```
MODULE PrimeChecker:
    Read A;
    B  <- A  - 1;
    IsPrime <- TRUE;
    WHILE   (B    != 1)
      DO  IF  (A/B gives no remainder)
            THEN  IsPrime <- FALSE;
        ENDIF;
     B  <-  B  - 1;
    ENDWHILE;
     RETURN IsPrime;
END.
```



It returns the variable is prime, and if the variable is prime is true it'll return true or if it'll false will return false.

Modularisation Approach

```
PROGRAM CheckPrime:
 IF (PrimeChecker = FALSE)
     THEN Print “Not Prime”;
     ELSE PRINT “Prime”;

 ENDIF;
 END.
```

Modularisation Approach

Modularisation make life easier for a lot of reasons:

- It's easier for someone else to understand how the code works
- It makes team programming a lot easier, different programmers can work on different methods
- Can improve the quality of the code
- Can reuse the same code over and over again(“don’t reinvent the wheel”)

Time for questions